Hookline: finding promos in microdrama screenplays

Lavanya L Nair · Kuku AI Generalist assignment · Problem 1, "Find the Promos" · App hookline-promos.streamlit.app · Code github.com/ayn-aval/hookline · Raw outputs outputs/i-married-a-broken-soldier

The problem I picked, and why

Today an editor watches every episode to pick promo moments, one show at a time, while new shows ship every week. I chose Problem 1 for three reasons:

- It is a real, recurring cost.
- Kuku gave real data.
- The output can be checked: every promo is a set of timestamps anyone can verify against the script.

Hookline reads a show's timestamped screenplays and returns a ranked list of ready-to-cut 20–60 s promos. For each promo it gives the episode and exact start/end of every segment, the hook, the cliffhanger, why it works, and why it sits at that rank. The editor's job shrinks from watching 50 episodes to reviewing about 20 suggestions.

Assumptions (where the brief was silent)

- **Script timestamps ≈ the final cut.** If the edit drifts, the timestamps shift with it.
- **The hook must be in the clip.** "First 3 seconds" means the hook beat starts ≤ 3.0 s after the promo starts.
- **Clip boundaries.** A cut may never split a spoken line. The cliffhanger is the promo's last beat.
- **Where viewers land.** A viewer who taps a promo starts at Episode 1. So, all else equal, a promo from Episode 3 converts better than one from Episode 40, whose payoff is 39 episodes away.
- **Spoilers.** Twists the show title already gives away ("He was a Duke") are fair game for ads. Later twists are penalised.
- **Budget.** Free tiers only. The model must be swappable, because Kuku may plug in stronger APIs.

How it works: what is AI, what is code, where the human is

```
screenplays → 1 Parse [code] → 2 Series bible [LLM ×1] → 3 Mine 5 episodes per call [LLM ×10]
            → 4 Stitch across moments [LLM ×1] → 5 Validate + repair [code] → 6 Cold-viewer judge [LLM ×3]
            → 7 Rank [code] → editor approves and exports [human]
```

- **Parse (code).** Every camera note, action, line and sound cue becomes a *beat* with an ID (`E07-B031`) and real timestamps. The pack has 50 episodes, 3,727 beats and 0 parse warnings.
- **LLM steps answer only with beat IDs, never timestamps,** so a model cannot invent a time. The **bible** maps aliases (e.g. "LADY IN GOLD GOWN" is Clarissa) and lists which reveals are ad-safe; **mining** proposes 2–4 windows per episode plus standout hook and cliffhanger moments; **stitch** joins 2–3 moments into one promo.
- **Validate (code) enforces the brief.** Unknown IDs are rejected, cuts are widened to whole lines, the start moves onto the hook and the end onto the cliffhanger. Windows over 60 s are condensed to the hook opening plus the run-up to the cliffhanger; short stitched promos get a longer run-up. Every repair and rejection is logged.
- **Judge (LLM, a different model).** It sees only the clip's own lines, with no series context, and scores hook, cliffhanger, standalone clarity and emotion from 1 to 5. This *tests* "makes sense on its own" instead of assuming it.
- **Rank (code).** Weighted judge score, +0.3 for 30–45 s, −0.6 per spoiler level, an early-episode preference, and no two promos from one episode back to back.
- **Human.** The editor reviews the ranked cards in the app, ticks Approve, and exports a CSV with one row per segment, ready to cut.

Models used for the committed run. `gemini-3.5-flash-lite` finds the promos and `gemini-3.1-flash-lite` judges them, both through Gemini's OpenAI-compatible API. Any OpenAI-compatible model is one flag away (`--provider`/`--model`/`--base-url`, or the app sidebar). I planned to use `gemini-3.5-flash`, but the free tier allows 20 requests per model per day, and my first per-episode design used them up while testing (see "AI tools" below).

Results on the input pack (50 episodes)

71 proposals were reduced to **56 promos** (35 continuous, 21 stitched): 41 were auto-repaired, 14 rejected (mostly too short), and 1 duplicate dropped. They cover 42 of the 50 episodes, with lengths of 20–60 s (median 43 s). Cross-episode stitching is supported, but in this run the lite model only stitched moments within one episode. The first three to post:

#	Segments	Hook (first 3 s)	Cliffhanger	Why it works
1	Ep 12 00:44.00–01:27.21	"I want to see you crawl, sister."	Juliana agrees to dig through frozen thorns to save her husband from the gallows	Cruel ultimatum, stakes spelled out in 40 s
2	Ep 13 01:20.80–01:58.88	"Stop!", then a scream in the briars	Bloodied hand closes on the ring	Raw physical sacrifice
3	Ep 1 00:11.00–01:05.60	"You should turn around, Juliana."	"You swore you loved me!"	Jilted at the altar for her sister; lands viewers on Ep 1

How I know it is good, and where it fails

I used three checks. The first two are judge scores (generator and judge are different models, and the judge is blind to which tool made a clip). The third is a manual read of the top 10.

Blind judge, mean 1–5	n	Hook	Cliff	Standalone	Weighted
Baseline: last 45 s of every episode	48	3.75	4.08	3.79	3.94
Baseline: best 15 by judge score	15	4.73	4.60	4.40	4.66
Hookline: all promos	56	3.80	3.70	3.66	3.80
Hookline: best 15 by judge score	15	4.87	4.53	4.07	4.60
Hookline: top 15 in posting order	15	4.73	4.33	4.07	4.51

What I read from this. The honest result is that the judge cannot separate Hookline's best from "cut the last 45 s".

- This show's episodes are *written* to end on cliffhangers, so the baseline wins on cliffhanger.
- Where Hookline measurably wins is the opening. **98% of Hookline promos have a spoken line in the first 3 s, against 79% of baseline cuts.** This check is objective; it needs no judge.
- Hookline also does things the baseline cannot: several promos per episode, stitched promos, a posting order, and spoiler control.
- **Stability:** a second model re-mining episodes 1–25 re-found 60% of the top-10 promos from those episodes.

Manual spot-check of the top 10 (script lines read against the brief, in `eval/spot_check.md`): **7 pass, 2 weak, 1 fail.** Every hook lands in the first 3 s, because code guarantees it. The failures are all things the model decides:

- **"Cliffhangers" that resolve.** #9 ends on "I've got you." and #2 ends as she finds the ring.
- **A lenient judge.** The lite judge gave 4–5 to almost everything, including those two clips.
- **Context-dependent clips.** #8 is a frame-up that needs the previous episode to make sense.
- **A spoiler paraphrase the keyword check missed.** #4, from Ep 39, shows the "dead" Duke returning.
- **Text only.** The tool never sees acting or footage, so a "hook" that is a camera move may not land.

Scale and cost

- **Per show (50 episodes):** 15 LLM calls, about 316K input and 26K output tokens, about 2 minutes. That is about **$0.16 per show** at paid Flash-Lite-class prices ($0.30 / $2.50 per 1M tokens, my assumption), and $0 on the free tier. 100 shows a week would cost about $16.
- **Re-runs and recovery:** every reply is cached in `raw/`, so re-runs and crash recovery are free. Changing a rule or weight in code re-ranks without calling any model.
- **On a paid tier:** mine one episode per call (`--episodes-per-call 1`) for the model's full attention, and use a stronger judge (`--judge-model`).
- **At Kuku's volume:** run each show as a batch job when its scripts land. Log editor approvals and rejections, and use them to tune the weights and prompts.

AI tools I used, and where they got it wrong

- **Claude Code** (Anthropic, Opus) to build: design, code, tests, deployment and drafting this doc. **Gemini** (Flash-Lite models) inside the tool. ChatGPT and Gemini chat were not used.
- **Wrong #1: the model can't add up timestamps.** Gemini Flash-Lite picked good moments but gave windows from 6 to 114 s, and 35 of 71 broke the 20–60 s rule. I moved all arithmetic into code (repair, condense, lengthen run-up), which brought rejections down to 14. Lesson: let the model judge drama and let code do the maths.
- **Wrong #2: the build plan didn't check rate limits.** Claude's first design made one LLM call per episode and never checked free-tier limits, so it used up the day's quota for `gemini-3.5-flash` mid-run. The fixes: batched calls, a separate judge model, and failing fast on a daily-quota error.

Next steps

A stronger judge, or pairwise "which would you post?" comparisons. A rule that flags endings whose last line resolves the tension. Editor-set ad-safe reveals per show. Audio and video signals (music stings, slaps) once footage is available.